Case Study: Agentic AI Intelligent Loan Approval System

Problem Statement

In modern banking systems, loan approval requires evaluating applicant details, credit history, risk indicators, and regulatory rules across multiple systems. Manual or rule-heavy approaches are slow, inconsistent, and difficult to scale. This case study focuses on designing and implementing a Multi-Agent Agentic AI system that analyzes loan applications and classifies them as Approved, Rejected, or Requires Manual Review.

Business Objective

- Automate loan application analysis using Agentic AI
- Improve decision speed and consistency
- Provide explainable, auditable decisions
- Adopt a scalable, loosely coupled microservices architecture

Input Parameters (Loan Application Data)

- Applicant ID
- Applicant Profile (Age, Income, Employment Type)
- Credit Score
- Loan Amount & Tenure
- Existing Liabilities
- Location
- Application Timestamp

Proposed Solution – Multi-Agent Architecture

The solution follows a distributed, microservices-based Agentic AI architecture with independent agents collaborating through an orchestration layer.

System Architecture Components

1. Presentation Layer
- Streamlit-based chatbot UI for loan application submission and status display

2. Microservice Layer
- FastAPI REST endpoints to receive and validate application data

3. Orchestration Layer

• LangGraph-based orchestration engine coordinating agent workflows
• Decision routing and state management

4. Agent Layer (Domain-Specific Agents)
• Each agent handles a dedicated responsibility in the loan evaluation lifecycle

5. Communication Layer
• MCP (Model Context Protocol) servers for standardized agent communication

Multi-Agent System Workflow

1. The user submits loan application data via the Streamlit chatbot UI
2. The UI forwards the data to a FastAPI-based microservice
3. The microservice passes the request to the Agentic AI Orchestration Engine
4. The orchestrator invokes relevant agents through MCP servers
5. Agents fetch contextual data, perform analysis, and return results
6. The orchestrator synthesizes outcomes and invokes the LLM
7. The final decision is propagated back to the chatbot UI

Agent Responsibilities Summary

1. Applicant Profile Agent
MCP Server: ApplicantDB
Output:
• Income Stability Score
• Employment Risk
• Credit History Summary
• Application Completeness Flags

2. Financial Risk Analysis Agent
MCP Server: RiskRulesDB
Output:
• Debt-to-Income Ratio
• Credit Score Risk Level
• Loan Amount Risk
• Anomaly Detection
• Reasoning

3. Loan Decision Agent
MCP Server: DecisionSynthesis
Output:
• Classification (Approve / Reject / Review)
• Risk Score

• Confidence Level
• Key Decision Factors
• Explanation

4. Compliance & Action Orchestrator Agent
MCP Server: NotificationSystem
Output:
• Action Taken
• Notification Sent
• Case ID
• Timestamp
• Summary

Technology Stack

• Chatbot UI: Streamlit (Python)
• Microservices: FastAPI (Python)
• Orchestration Engine: LangGraph, LangChain
• MCP Framework: FastMCP
• Agent Implementation: FastAPI-based agents
• LLM: Anthropic Claude Sonnet 4.6
• Agent SDK: Anthropic Agent SDK
• Prompt Engineering
• Claude Code
• Python 3.x

Evaluation Criteria

Each participant must submit their own implementation and demonstrate:

• Understanding of Agentic AI architecture
• Correct orchestration using LangGraph
• Clear agent responsibilities and MCP usage
• Ability to modify code live during evaluation
• Explainable AI outputs

Evaluation Process

• Internal technical evaluation via one-on-one session
• Live code walkthrough is needed during discussion
• Relative performance scoring